Dimensionality Reduction for Face Classification Using PCA and LDA

JI YANCHEN

EE6222 - Assignment 1

Abstract

This study evaluates how representation dimension affects Euclidean 1-nearest-neighbour (1-NN) classification on Olivetti Faces. A nested 5-fold outer / 4-fold inner design selects dimensions using outer-training data and scores each selected model once on its outer fold. Across 400 out-of-fold predictions, the mean fold accuracy is 0.9400 for raw pixels, 0.9350 for inner-selected PCA, 0.9875 for direct LDA, and 0.9850 for inner-selected PCA-to-LDA. The earlier PCA280-to-LDA collapse is metric-sensitive: training-fitted standardization of LDA coordinates raises its outer-fold accuracy from 0.5000 to 0.9375 at 20 outputs. PCA319-to-LDA remains lower; its within-class SVD rank is 280 of 319, and the solver truncates 39 directions containing about 90% of standardized class-center energy. These results are specific to this dataset and classifier.

1. Introduction

A 64 × 64 face image becomes a 4096-dimensional pixel vector. PCA preserves directions of high total variance, whereas LDA uses identity labels to construct class-discriminative coordinates [1, 2]. Their interaction matters to a distance-based classifier: 1-NN is sensitive to both the retained subspace and coordinate scaling.

This assignment compares raw pixels, PCA, direct LDA, and PCA followed by LDA with one fixed classifier. The analysis focuses on dimension curves and investigates why high-dimensional PCA-to-LDA settings performed poorly under the original Euclidean metric. Mechanisms are treated as interpretations of this experiment, not universal claims.

2. Experimental Methods

Dataset and preprocessing

The scikit-learn Olivetti loader supplies 400 grayscale images from 40 identities, 10 images per identity, each flattened to 4096 values in [0, 1] [3]. The original Database of Faces page requests credit to AT&T Laboratories Cambridge [6]. The public program downloads the data through scikit-learn when the cache is absent. All fitting uses training samples; PCA centers its training input. The classifier is Euclidean 1-NN (k = 1). LDA uses scikit-learn 1.7.2 with `solver="svd"` and `tol=1e-4` [4].

Nested selection and evaluation

The fixed outer partition is a shuffled stratified 5-fold split (`random_state=20261012`). Each outer fold has 320 training and 80 test images, with 8 and 2 images per identity; the five test folds partition the 400 images exactly once. Within each outer training fold, shuffled stratified 4-fold CV (`random_state=20261020 + fold`) selects dimensions by mean validation accuracy. Ties prefer the smaller output dimension, then the smaller PCA pre-dimension. The selected pipeline is refitted on the full outer training fold and evaluated on its outer test fold. This follows the train-only model-selection / held-out scoring separation described in standard cross-validation practice [5].

PCA candidates are 1, 2, 5, 10, 20, 30, 40, 50, 75, 100, 150, and 200. Direct LDA and PCA-to-LDA final dimensions are 1, 2, 5, 10, 15, 20, 25, 30, 35, and 39. PCA-to-LDA pre-dimensions 50, 75, 100, 150, and 200 are eligible for inner selection. PCA280 and PCA319 are fixed diagnostic controls, not selected from outer-test scores.

Curves, controls, and scope

Every fixed dimension is fitted on an outer training fold and scored on its outer test fold to create performance curves. Error rate is 1 - accuracy. The LDA-coordinate normalization control fits a `StandardScaler` on training outputs only, then applies it to train and test outputs before 1-NN. It is a post-hoc diagnostic prompted by the observed scale imbalance, not part of dimension selection or the selected-model comparison. Tables and curves report the mean and sample SD across five outer folds. These folds reuse one dataset and their SD is descriptive, not a confidence interval or variation across independent datasets.

The previous v1 experiment used the seed-42 test curve to choose representative dimensions, then repartitioned the same 400 images for seeds 100-119. Those v1 repeated-holdout numbers are retained as historical evidence,

not treated as independent confirmation. This v2 analysis uses nested selection per outer fold, but follows that earlier exploration and has no external sample-level test set.

3. Results

Method	Inner-selected settings across folds	Outer mean accuracy ± SD
Raw 1-NN	4096 pixels	0.9400 ± 0.0311
PCA + 1-NN	40 (2 folds), 75, 150, 200	0.9350 ± 0.0298
Direct LDA + 1-NN	25, 30 (2), 35, 39 outputs	0.9875 ± 0.0217
PCA-to-LDA + 1-NN	PCA 50/75/100; LDA 30/35/39	0.9850 ± 0.0224

Table 1. Each selected-model mean aggregates five outer folds (80 test predictions per fold; 400 out-of-fold predictions total). Direct LDA classified 395/400 images correctly and selected PCA-to-LDA classified 394/400. The one-prediction difference is descriptive; five folds do not support a precise ranking between these two settings.

PCA accuracy rises quickly at low dimension: 0.7450 at 5 components, 0.8875 at 10, and 0.9350 at 20. It then plateaus between 0.9350 and 0.9425 from 20 to 200 components; the peak is 0.9425 at 75 and 100. Thus, most of the observed gain occurs by 20 components, and adding components does not produce a monotonic improvement.

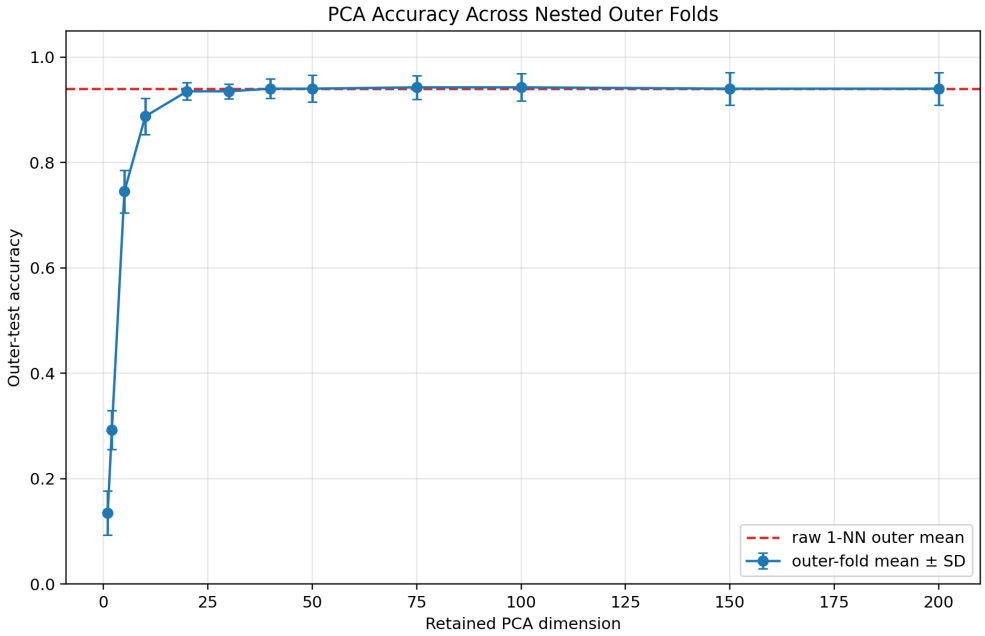


Figure 1. Outer-fold mean PCA accuracy by retained dimension; error bars show ±1 sample SD across five folds. The dashed line is the raw-pixel mean.

3. Results (continued)

Direct LDA rises from 0.1850 at one output to 0.9775 at 10, then stays between 0.9825 and 0.9875 from 15 to 39. Its use of labels is consistent with stronger identity separation than variance-only PCA on this dataset. The inner-selected PCA-to-LDA models also score highly, averaging 0.9850; the earlier poor results therefore apply to the tested high pre-dimensions and raw Euclidean geometry, not to every PCA-to-LDA configuration.

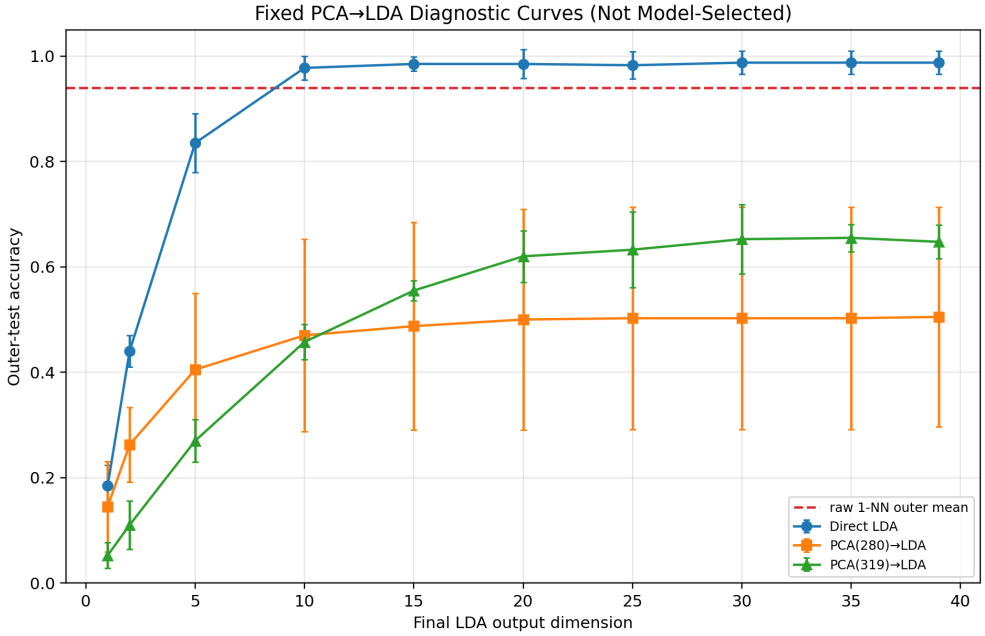


Figure 2. Outer-fold mean accuracy versus final LDA output dimension for direct LDA, fixed PCA280-to-LDA and PCA319-to-LDA controls. Error bars show ±1 SD; horizontal dashed line is raw 1-NN.

At PCA280-to-LDA20, raw Euclidean 1-NN averages 0.5000 ± 0.2095 across outer folds. Standardizing each LDA coordinate from outer-training outputs raises this to 0.9375 ± 0.0319; at 39 outputs it reaches 0.9725 ± 0.0205. PCA319-to-LDA25 averages 0.6325 ± 0.0716, and the same coordinate standardization yields 0.6250 ± 0.0884. The metric control strongly supports a coordinate-scale effect for PCA280 and does not explain the PCA319 result.

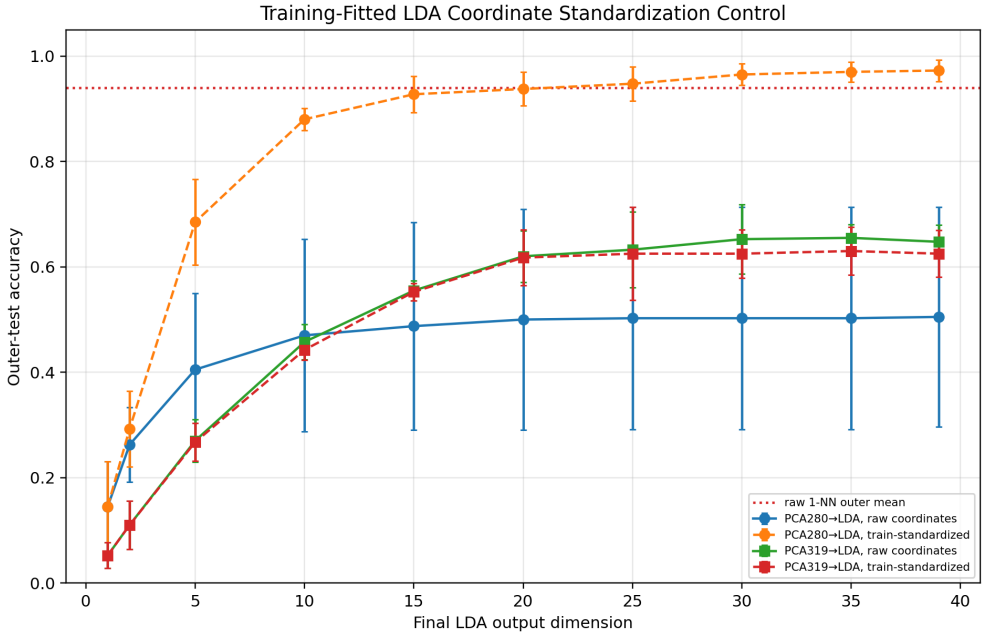


Figure 3. Fixed PCA280/319-to-LDA curves before and after LDA-output standardization fitted on outer-training data only. The dashed line is the raw 1-NN outer-fold mean.

4. Discussion

Why the high-dimensional PCA-to-LDA controls differ

For 320 training samples and 40 classes, the within-class scatter rank is at most 320 - 40 = 280. Scikit-learn's SVD solver scales within-class-centered features by their within-class standard deviation, keeps within singular values strictly above the absolute threshold 1e-4, then performs a second SVD on class centers with a relative threshold [4]. PCA280 is full within-rank: the seed-42 fit retains 280/280 directions, with retained-spectrum condition number 265.3. Across the five outer folds, the condition ranges from 503 to 3472. LDA output coordinate scales are unequal (median max/min SD ratio 15.2). In the 40-case, one-per-identity internal check, the median nearest-same / nearest-different distance ratio changes from 0.619 to 0.485 after standardization; PCA319 remains near 1.000 before and after. Training-fitted coordinate standardization restores PCA280-to-LDA accuracy, so ordinary Euclidean distance on raw outputs is a substantial part of its collapse. It does not establish that scaling is the only relevant mechanism. Because the scale control followed inspection of the fixed curves, its accuracy is exploratory rather than confirmatory.

PCA319 has 319 input features but still at most 280 within-class directions. The SVD solver retains 280 and truncates 39. In the seed-42 training fit, 90.1% of the standardized class-center squared energy lies in those discarded directions; the five outer fits range from 89.9% to 90.5%. This provides a data-specific mechanism for losing useful between-class structure. Its retained within spectrum is well-conditioned (condition 1.62), LDA output scales are nearly balanced (median ratio 1.24), and coordinate standardization does not recover accuracy. The result is therefore consistent with rank-truncation loss, rather than a simple output-scale imbalance. The 319-dimensional leave-one-out diagnostic is additionally a boundary stress case: each inner fit has 319 rows, so centered PCA has rank at most 318 and its last component is near zero. That near-chance LOO score is not used as a test estimate and does not explain the less severe 0.63 outer-fold result by itself.

Evaluation limits and conclusion

The original v1 representative settings were selected from seed-42 test accuracy, which is performance-guided dimension selection. New seeds 100-119 changed the split, not the image sample; they are repeated holdouts, not an independent test set. In v2, each outer test fold is used only for the fixed performance curves and the selected-model score, while its dimensions are chosen inside the outer training data. The earlier seed-42 exploration informed this follow-up, and the scale and nullspace analyses were also post-hoc diagnostics on the same image cohort. The results are a transparent cross-validation study, not an independent replication or causal confirmation.

On this dataset and classifier, direct LDA and inner-selected PCA-to-LDA both approach 99% accuracy, while selected PCA stays near raw 1-NN. The earlier high-dimensional PCA-to-LDA collapse is explained in part by unequal LDA-coordinate scales at PCA280 and by SVD truncation of within-null directions at PCA319. It is not evidence that PCA-to-LDA is generally inferior.

References

[1] X. Jiang, "Linear Subspace Learning-Based Dimensionality Reduction," *IEEE Signal Processing Magazine*, vol. 28, no. 2, pp. 16-26, 2011. doi: 10.1109/MSP.2010.939041.

[2] P. N. Belhumeur, J. P. Hespanha, and D. J. Kriegman, "Eigenfaces vs. Fisherfaces: Recognition Using Class Specific Linear Projection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 19, no. 7, pp. 711-720, 1997. doi: 10.1109/34.598228.

[3] Scikit-learn developers, `fetch_olivetti_faces` API reference, version 1.7.2, accessed 12 Oct. 2026: scikit-learn.org/1.7/modules/generated/sklearn.datasets.fetch_olivetti_faces.html.

[4] Scikit-learn developers, `LinearDiscriminantAnalysis` API reference, version 1.7.2, accessed 12 Oct. 2026: scikit-learn.org/1.7/modules/generated/sklearn.discriminant_analysis.LinearDiscriminantAnalysis.html.

[5] Scikit-learn developers, "Cross-validation: evaluating estimator performance," version 1.7.2, accessed 12 Oct. 2026: scikit-learn.org/1.7/modules/cross_validation.html.

[6] AT&T Laboratories Cambridge, "The Database of Faces," accessed 13 Oct. 2026: https://www.cl.cam.ac.uk/research/dtg/attarchive/facedatabase.html.

Appendix A. Complete Executable Program

This complete program loads Olivetti Faces through scikit-learn (using an existing cache or downloading it when absent), runs nested dimension selection and fixed-curve experiments, computes accuracy and error rates, aggregates outer-fold means and standard deviations, and writes tables, figures, a manifest, and validation results. The public repository provides this standalone script as its reproducible program.

EE6222_Assignment_Standalone.py

```
from __future__ import annotations

import hashlib
import json
import logging
import os
import socket
import sys
import time
from collections import Counter
from pathlib import Path
from typing import Any

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import scipy
from sklearn.datasets import fetch_olivetti_faces
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.metrics import accuracy_score
from sklearn.model_selection import StratifiedKFold
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler

ROOT = Path(__file__).resolve().parent
OUTPUT = ROOT / "results" / "second_pass"
AUDIT = ROOT / "audit" / "second_pass"
LOGS = ROOT / "results" / "logs"
PCA_DIMS = (1, 2, 5, 10, 20, 30, 40, 50, 75, 100, 150, 200)
LDA_DIMS = (1, 2, 5, 10, 15, 20, 25, 30, 35, 39)
PCA_LDA_PRE_DIMS = (50, 75, 100, 150, 200)
PCA_LDA_DIAGNOSTIC_PRE_DIMS = (280, 319)
OUTER_FOLDS = 5
INNER_FOLDS = 4
OUTER_SEED = 20261012
INNER_SEED_BASE = 20261020
LDA_SOLVER = "svd"
LDA_TOL = 1e-4
KNN_NEIGHBORS = 1
KNN_METRIC = "euclidean"
RESULT_COLUMNS = (
    "outer_fold",
    "method",
    "pca_pre_dimension",
    "output_dimension",
    "accuracy",
    "error_rate",
    "n_correct",
    "n_test",
    "inner_selection_score",
)


def configure_logging(path: Path) -> logging.Logger:
    path.parent.mkdir(parents=True, exist_ok=True)
    logger = logging.getLogger("ee6222.standalone")
    logger.handlers.clear()
    logger.setLevel(logging.INFO)
    logger.propagate = False
    formatter = logging.Formatter("%(asctime)s | %(levelname)s | %(message)s")
    for handler in (
        logging.FileHandler(path, mode="w", encoding="utf-8"),
        logging.StreamHandler(sys.stdout),
    ):
        handler.setFormatter(formatter)
        logger.addHandler(handler)
    return logger


def sha256_bytes(payload: bytes) -> str:
    return hashlib.sha256(payload).hexdigest()


def sha256_file(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for chunk in iter(lambda: handle.read(1024 * 1024), b""):
            digest.update(chunk)
    return digest.hexdigest()


def write_json(path: Path, value: Any) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(
        json.dumps(value, indent=2, ensure_ascii=False, sort_keys=True, default=str) + "\n",
        encoding="utf-8",
    )
```

```
|
|
| def load_dataset() -> tuple[np.ndarray, np.ndarray, dict[str, Any]]:
|     cache = ROOT / "data" / "sklearn_cache"
|     dataset = fetch_olivetti_faces(
|         data_home=str(cache),
|         shuffle=False,
|         download_if_missing=True,
|     )
|     X = np.asarray(dataset.data)
|     y = np.asarray(dataset.target)
|     if X.shape != (400, 4096) or y.shape != (400,):
|         raise RuntimeError(f"Unexpected Olivetti shape: X={X.shape}, y={y.shape}")
|     labels, counts = np.unique(y, return_counts=True)
|     if labels.size != 40 or not np.all(counts == 10):
|         raise RuntimeError("Expected 40 identities with 10 images each")
|     if not np.isfinite(X).all():
|         raise RuntimeError("Dataset contains non-finite features")
|     audit = {
|         "source": "sklearn.datasets.fetch_olivetti_faces",
|         "data_home": "data/sklearn_cache",
|         "X_shape": list(X.shape),
|         "y_shape": list(y.shape),
|         "X_dtype": str(X.dtype),
|         "y_dtype": str(y.dtype),
|         "class_count": int(labels.size),
|         "samples_per_class": {str(label): int(count) for label, count in zip(labels, counts)},
|         "X_sha256": sha256_bytes(np.ascontiguousarray(X).tobytes()),
|         "y_sha256": sha256_bytes(np.ascontiguousarray(y).tobytes()),
|     }
|     return X, y, audit
|
|
| def fit_predict_knn(
|     X_train: np.ndarray,
|     y_train: np.ndarray,
|     X_valid: np.ndarray,
| ) -> np.ndarray:
|     model = KNeighborsClassifier(n_neighbors=KNN_NEIGHBORS, metric=KNN_METRIC)
|     model.fit(X_train, y_train)
|     return model.predict(X_valid)
|
|
| def evaluate_accuracy(
|     X_train: np.ndarray,
|     y_train: np.ndarray,
|     X_valid: np.ndarray,
|     y_valid: np.ndarray,
| ) -> float:
|     return float(accuracy_score(y_valid, fit_predict_knn(X_train, y_train, X_valid)))
|
|
| def inner_select(
|     X_outer_train: np.ndarray,
|     y_outer_train: np.ndarray,
|     outer_fold: int,
|     logger: logging.Logger,
| ) -> tuple[dict[str, Any], list[dict[str, Any]]]:
|     """Select dimensions using only the current outer-training fold."""
|     inner_cv = StratifiedKFold(
|         n_splits=INNER_FOLDS,
|         shuffle=True,
|         random_state=INNER_SEED_BASE + outer_fold,
|     )
|     pca_candidates = sorted(set(PCA_DIMS) | set(PCA_LDA_PRE_DIMS))
|     pca_scores: dict[int, list[float]] = {d: [] for d in pca_candidates}
|     lda_scores: dict[int, list[float]] = {d: [] for d in LDA_DIMS}
|     pca_lda_scores: dict[tuple[int, int], list[float]] = {
|         (pre, output): []
|         for pre in PCA_LDA_PRE_DIMS
|         for output in LDA_DIMS
|     }
|
|     for inner_fold, (train_idx, valid_idx) in enumerate(
|         inner_cv.split(X_outer_train, y_outer_train), start=1
|     ):
|         X_train, X_valid = X_outer_train[train_idx], X_outer_train[valid_idx]
|         y_train, y_valid = y_outer_train[train_idx], y_outer_train[valid_idx]
|         if X_train.shape[0] != 240 or X_valid.shape[0] != 80:
|             raise RuntimeError("Unexpected 4-fold inner split sizes")
|         if np.unique(y_valid).size != 40 or not np.all(
|             np.unique(y_valid, return_counts=True)[1] == 2
|         ):
|             raise RuntimeError("Inner validation fold must contain 2 samples per identity")
|
|         projected: dict[int, tuple[np.ndarray, np.ndarray]] = {}
|         for dimension in pca_candidates:
|             if dimension > min(X_train.shape):
|                 raise ValueError(f"Inner PCA dimension {dimension} is illegal")
|             pca = PCA(n_components=dimension, svd_solver="full")
|             pca.fit(X_train)
|             projected[dimension] = (pca.transform(X_train), pca.transform(X_valid))
|
|         for dimension in PCA_DIMS:
|             Xtr, Xva = projected[dimension]
|             pca_scores[dimension].append(
|                 evaluate_accuracy(Xtr, y_train, Xva, y_valid)
|             )
|
|         for dimension in LDA_DIMS:
|             lda = LinearDiscriminantAnalysis(
|                 solver=LDA_SOLVER,
```

```
                n_components=dimension,
                tol=LDA_TOL,
            )
            lda.fit(X_train, y_train)
            Ztr = lda.transform(X_train)
            Zva = lda.transform(X_valid)
            lda_scores[dimension].append(
                evaluate_accuracy(Ztr, y_train, Zva, y_valid)
            )

        for pre_dimension in PCA_LDA_PRE_DIMS:
            Xtr_pca, Xva_pca = projected[pre_dimension]
            for output_dimension in LDA_DIMS:
                lda = LinearDiscriminantAnalysis(
                    solver=LDA_SOLVER,
                    n_components=output_dimension,
                    tol=LDA_TOL,
                )
                lda.fit(Xtr_pca, y_train)
                Ztr = lda.transform(Xtr_pca)
                Zva = lda.transform(Xva_pca)
                pca_lda_scores[(pre_dimension, output_dimension)].append(
                    evaluate_accuracy(Ztr, y_train, Zva, y_valid)
                )
        logger.info(
            "outer=%d inner=%d/%d selection folds complete",
            outer_fold,
            inner_fold,
            INNER_FOLDS,
        )

    candidate_rows: list[dict[str, Any]] = []
    for dimension, scores in pca_scores.items():
        candidate_rows.append(
            {
                "outer_fold": outer_fold,
                "family": "pca",
                "pca_pre_dimension": None,
                "output_dimension": dimension,
                "mean_inner_accuracy": float(np.mean(scores)),
                "sd_inner_accuracy": float(np.std(scores, ddof=1)),
            }
        )
    for dimension, scores in lda_scores.items():
        candidate_rows.append(
            {
                "outer_fold": outer_fold,
                "family": "direct_lda",
                "pca_pre_dimension": None,
                "output_dimension": dimension,
                "mean_inner_accuracy": float(np.mean(scores)),
                "sd_inner_accuracy": float(np.std(scores, ddof=1)),
            }
        )
    for (pre_dimension, output_dimension), scores in pca_lda_scores.items():
        candidate_rows.append(
            {
                "outer_fold": outer_fold,
                "family": "pca_lda",
                "pca_pre_dimension": pre_dimension,
                "output_dimension": output_dimension,
                "mean_inner_accuracy": float(np.mean(scores)),
                "sd_inner_accuracy": float(np.std(scores, ddof=1)),
            }
        )

    selected_pca = min(
        pca_scores,
        key=lambda d: (-float(np.mean(pca_scores[d])), d),
    )
    selected_lda = min(
        lda_scores,
        key=lambda d: (-float(np.mean(lda_scores[d])), d),
    )
    selected_pca_lda = min(
        pca_lda_scores,
        key=lambda pair: (
            -float(np.mean(pca_lda_scores[pair])),
            pair[1],
            pair[0],
        ),
    )
    selected = {
        "pca_dimension": int(selected_pca),
        "pca_inner_accuracy": float(np.mean(pca_scores[selected_pca])),
        "direct_lda_dimension": int(selected_lda),
        "direct_lda_inner_accuracy": float(np.mean(lda_scores[selected_lda])),
        "pca_lda_pre_dimension": int(selected_pca_lda[0]),
        "pca_lda_output_dimension": int(selected_pca_lda[1]),
        "pca_lda_inner_accuracy": float(np.mean(pca_lda_scores[selected_pca_lda])),
    }
    return selected, candidate_rows


def outer_curve_rows(
    fold: int,
    X_train: np.ndarray,
    y_train: np.ndarray,
    X_test: np.ndarray,
    y_test: np.ndarray,
) -> list[dict[str, Any]]:
    rows: list[dict[str, Any]] = []
```

```
    def append(method: str, pre: int | None, output: int, accuracy: float) -> None:
        rows.append(
            {
                "outer_fold": fold,
                "method": method,
                "pca_pre_dimension": pre,
                "output_dimension": int(output),
                "accuracy": float(accuracy),
                "error_rate": float(1.0 - accuracy),
                "n_correct": int(round(accuracy * y_test.size)),
                "n_test": int(y_test.size),
                "inner_selection_score": None,
            }
        )

    append("raw_1nn", None, X_train.shape[1], evaluate_accuracy(X_train, y_train, X_test, y_test))

    for dimension in PCA_DIMS:
        pca = PCA(n_components=dimension, svd_solver="full")
        pca.fit(X_train)
        Xtr = pca.transform(X_train)
        Xte = pca.transform(X_test)
        append("pca_curve", None, dimension, evaluate_accuracy(Xtr, y_train, Xte, y_test))

    for dimension in LDA_DIMS:
        lda = LinearDiscriminantAnalysis(solver=LDA_SOLVER, n_components=dimension, tol=LDA_TOL)
        lda.fit(X_train, y_train)
        Ztr = lda.transform(X_train)
        Zte = lda.transform(X_test)
        append("direct_lda_curve", None, dimension, evaluate_accuracy(Ztr, y_train, Zte, y_test))

    # These are fixed diagnostic controls from the earlier project analysis.
    # They are not searched or selected using these outer-fold scores.
    for pre_dimension in PCA_LDA_DIAGNOSTIC_PRE_DIMS:
        pca = PCA(n_components=pre_dimension, svd_solver="full")
        pca.fit(X_train)
        Xtr_pca = pca.transform(X_train)
        Xte_pca = pca.transform(X_test)
        if pre_dimension == 280:
            append(
                "pca280_only_curve",
                pre_dimension,
                pre_dimension,
                evaluate_accuracy(Xtr_pca, y_train, Xte_pca, y_test),
            )
        for dimension in LDA_DIMS:
            lda = LinearDiscriminantAnalysis(solver=LDA_SOLVER, n_components=dimension, tol=LDA_TOL)
            lda.fit(Xtr_pca, y_train)
            Ztr = lda.transform(Xtr_pca)
            Zte = lda.transform(Xte_pca)
            append(
                f"pca{pre_dimension}_lda_curve",
                pre_dimension,
                dimension,
                evaluate_accuracy(Ztr, y_train, Zte, y_test),
            )
            # Diagnostic only: fit coordinate standardization on outer-train outputs.
            scaler = StandardScaler().fit(Ztr)
            Ztr_scaled = scaler.transform(Ztr)
            Zte_scaled = scaler.transform(Zte)
            append(
                f"pca{pre_dimension}_lda_trainstd_curve",
                pre_dimension,
                dimension,
                evaluate_accuracy(Ztr_scaled, y_train, Zte_scaled, y_test),
            )
    return rows


def fit_selected_outer(
    fold: int,
    selected: dict[str, Any],
    X_train: np.ndarray,
    y_train: np.ndarray,
    X_test: np.ndarray,
    y_test: np.ndarray,
) -> list[dict[str, Any]]:
    selected_rows: list[dict[str, Any]] = []

    def append(method: str, pre: int | None, dim: int, score: float, inner: float) -> None:
        selected_rows.append(
            {
                "outer_fold": fold,
                "method": method,
                "pca_pre_dimension": pre,
                "output_dimension": int(dim),
                "accuracy": float(score),
                "error_rate": float(1.0 - score),
                "n_correct": int(round(score * y_test.size)),
                "n_test": int(y_test.size),
                "inner_selection_score": float(inner),
            }
        )

    pca_dim = selected["pca_dimension"]
    pca = PCA(n_components=pca_dim, svd_solver="full").fit(X_train)
    append(
        "pca_selected",
        None,
        pca_dim,
        evaluate_accuracy(pca.transform(X_train), y_train, pca.transform(X_test), y_test),
```

```
        selected["pca_inner_accuracy"],
    )

    lda_dim = selected["direct_lda_dimension"]
    lda = LinearDiscriminantAnalysis(solver=LDA_SOLVER, n_components=lda_dim, tol=LDA_TOL).fit(X_train, y_train)
    append(
        "direct_lda_selected",
        None,
        lda_dim,
        evaluate_accuracy(lda.transform(X_train), y_train, lda.transform(X_test), y_test),
        selected["direct_lda_inner_accuracy"],
    )

    pre_dim = selected["pca_lda_pre_dimension"]
    out_dim = selected["pca_lda_output_dimension"]
    pca = PCA(n_components=pre_dim, svd_solver="full").fit(X_train)
    Xtr_pca, Xte_pca = pca.transform(X_train), pca.transform(X_test)
    lda = LinearDiscriminantAnalysis(solver=LDA_SOLVER, n_components=out_dim, tol=LDA_TOL).fit(Xtr_pca, y_train)
    append(
        "pca_lda_selected",
        pre_dim,
        out_dim,
        evaluate_accuracy(lda.transform(Xtr_pca), y_train, lda.transform(Xte_pca), y_test),
        selected["pca_lda_inner_accuracy"],
    )
    return selected_rows


def aggregate_curves(frame: pd.DataFrame) -> pd.DataFrame:
    keys = ["method", "pca_pre_dimension", "output_dimension"]
    return (
        frame.groupby(keys, dropna=False, sort=True)
        .agg(
            n_outer_folds=("outer_fold", "nunique"),
            mean_accuracy=("accuracy", "mean"),
            sd_accuracy=("accuracy", "std"),
            mean_error_rate=("error_rate", "mean"),
            min_accuracy=("accuracy", "min"),
            max_accuracy=("accuracy", "max"),
        )
        .reset_index()
    )


def paired_fold_comparisons(selected: pd.DataFrame) -> pd.DataFrame:
    pivot = selected.pivot(index="outer_fold", columns="method", values="accuracy")
    rows = []
    for method in ("pca_selected", "direct_lda_selected", "pca_lda_selected"):
        delta = pivot[method] - pivot["raw_1nn"]
        rows.append(
            {
                "comparison": f"{method} - raw_1nn",
                "n_outer_folds": int(delta.size),
                "mean_delta": float(delta.mean()),
                "wins": int((delta > 0).sum()),
                "ties": int((delta == 0).sum()),
                "losses": int((delta < 0).sum()),
            }
        )
    return pd.DataFrame(rows)


def save_figures(curves: pd.DataFrame, selected: pd.DataFrame, out_dir: Path) -> None:
    out_dir.mkdir(parents=True, exist_ok=True)
    raw_mean = float(selected[selected.method == "raw_1nn"]["accuracy"].mean())
    for method, title, filename in (
        ("pca_curve", "PCA Accuracy Across Nested Outer Folds", "pca_accuracy_vs_dimension_nested.png"),
        ("direct_lda_curve", "Direct LDA Accuracy Across Nested Outer Folds", "lda_accuracy_vs_dimension_nested.png"),
    ):
        subset = curves[curves.method == method].sort_values("output_dimension")
        fig, ax = plt.subplots(figsize=(8.5, 5.5), constrained_layout=True)
        ax.errorbar(
            subset.output_dimension,
            subset.mean_accuracy,
            yerr=subset.sd_accuracy.fillna(0.0),
            marker="o",
            capsize=3,
            label="outer-fold mean ± SD",
        )
        ax.axhline(raw_mean, color="tab:red", linestyle="--", label="raw 1-NN outer mean")
        ax.set_xlabel("Retained PCA dimension" if method == "pca_curve" else "Final LDA output dimension")
        ax.set_ylabel("Outer-test accuracy")
        ax.set_ylim(0.0, 1.05)
        ax.set_title(title)
        ax.grid(True, alpha=0.3)
        ax.legend()
        fig.savefig(out_dir / filename, dpi=220)
        plt.close(fig)

    pca280 = curves[curves.method == "pca280_lda_curve"].sort_values("output_dimension")
    pca319 = curves[curves.method == "pca319_lda_curve"].sort_values("output_dimension")
    direct = curves[curves.method == "direct_lda_curve"].sort_values("output_dimension")
    fig, ax = plt.subplots(figsize=(8.5, 5.5), constrained_layout=True)
    for table, label, marker in (
        (direct, "Direct LDA", "o"),
        (pca280, "PCA(280)→LDA", "s"),
        (pca319, "PCA(319)→LDA", "^"),
    ):
        ax.errorbar(
            table.output_dimension,
            table.mean_accuracy,
            yerr=table.sd_accuracy.fillna(0.0),
```

```
            marker=marker,
            capsize=2,
            label=label,
        )
    ax.axhline(raw_mean, color="tab:red", linestyle="--", label="raw 1-NN outer mean")
    ax.set_xlabel("Final LDA output dimension")
    ax.set_ylabel("Outer-test accuracy")
    ax.set_ylim(0.0, 1.05)
    ax.set_title("Fixed PCA→LDA Diagnostic Curves (Not Model-Selected)")
    ax.grid(True, alpha=0.3)
    ax.legend(fontsize=8)
    fig.savefig(out_dir / "pca_lda_fixed_controls_nested.png", dpi=220)
    plt.close(fig)

    fig, ax = plt.subplots(figsize=(8.5, 5.5), constrained_layout=True)
    for method, label, marker, linestyle in (
        ("pca280_lda_curve", "PCA280→LDA, raw coordinates", "o", "-"),
        ("pca280_lda_trainstd_curve", "PCA280→LDA, train-standardized", "o", "--"),
        ("pca319_lda_curve", "PCA319→LDA, raw coordinates", "s", "-"),
        ("pca319_lda_trainstd_curve", "PCA319→LDA, train-standardized", "s", "--"),
    ):
        subset = curves[curves.method == method].sort_values("output_dimension")
        ax.errorbar(
            subset.output_dimension,
            subset.mean_accuracy,
            yerr=subset.sd_accuracy.fillna(0.0),
            marker=marker,
            linestyle=linestyle,
            capsize=2,
            label=label,
        )
    ax.axhline(raw_mean, color="tab:red", linestyle=":", label="raw 1-NN outer mean")
    ax.set_xlabel("Final LDA output dimension")
    ax.set_ylabel("Outer-test accuracy")
    ax.set_ylim(0.0, 1.05)
    ax.set_title("Training-Fitted LDA Coordinate Standardization Control")
    ax.grid(True, alpha=0.3)
    ax.legend(fontsize=7)
    fig.savefig(out_dir / "pca_lda_geometry_normalization_nested.png", dpi=220)
    plt.close(fig)

    stats = selected.groupby("method", sort=False).accuracy.agg(["mean", "std"]).reindex(
        ["raw_1nn", "pca_selected", "direct_lda_selected", "pca_lda_selected"]
    )
    labels = ["Raw", "PCA selected", "Direct LDA selected", "PCA→LDA selected"]
    fig, ax = plt.subplots(figsize=(8.5, 5.5), constrained_layout=True)
    ax.errorbar(
        np.arange(len(stats)),
        stats["mean"],
        yerr=stats["std"].fillna(0.0),
        fmt="o",
        capsize=4,
        color="tab:blue",
    )
    ax.set_xticks(np.arange(len(stats)), labels, rotation=10)
    ax.set_ylabel("Outer-test accuracy")
    ax.set_ylim(0.0, 1.05)
    ax.set_title("Nested Model Selection (Mean ± SD Across Five Outer Folds)")
    ax.grid(True, axis="y", alpha=0.3)
    fig.savefig(out_dir / "nested_selected_model_comparison.png", dpi=220)
    plt.close(fig)


def run() -> int:
    OUTPUT.mkdir(parents=True, exist_ok=True)
    AUDIT.mkdir(parents=True, exist_ok=True)
    logger = configure_logging(LOGS / "standalone_nested_run.log")
    started = time.perf_counter()
    X, y, data_audit = load_dataset()

    outer_cv = StratifiedKFold(
        n_splits=OUTER_FOLDS,
        shuffle=True,
        random_state=OUTER_SEED,
    )
    outer_splits = list(outer_cv.split(X, y))
    outer_test_indices = [np.asarray(test, dtype=np.int64) for _, test in outer_splits]
    test_union = np.concatenate(outer_test_indices)
    if test_union.size != X.shape[0] or np.unique(test_union).size != X.shape[0]:
        raise RuntimeError("Outer test folds must partition all 400 samples exactly once")

    all_outer_rows: list[dict[str, Any]] = []
    selected_rows: list[dict[str, Any]] = []
    selection_rows: list[dict[str, Any]] = []
    fold_records: list[dict[str, Any]] = []

    for fold, (train_idx, test_idx) in enumerate(outer_splits, start=1):
        X_train, X_test = X[train_idx], X[test_idx]
        y_train, y_test = y[train_idx], y[test_idx]
        if X_train.shape[0] != 320 or X_test.shape[0] != 80:
            raise RuntimeError(f"Outer fold {fold} has unexpected sizes")
        if not np.array_equal(np.unique(y_test, return_counts=True)[1], np.repeat(2, 40)):
            raise RuntimeError(f"Outer fold {fold} is not stratified 8/2 per class")
        logger.info("outer fold %d/%d: begin inner train-only selection", fold, OUTER_FOLDS)
        selected, candidates = inner_select(X_train, y_train, fold, logger)
        selection_rows.extend(candidates)
        fold_records.append(
            {
                "outer_fold": fold,
                "train_indices": train_idx.tolist(),
                "test_indices": test_idx.tolist(),
                "train_index_sha256": sha256_bytes(np.asarray(train_idx, dtype=np.int64).tobytes()),
```

```
                "test_index_sha256": sha256_bytes(np.asarray(test_idx, dtype=np.int64).tobytes()),
                "selected": selected,
            }
        )
        logger.info("outer fold %d selected: %s", fold, selected)

        all_outer_rows.extend(outer_curve_rows(fold, X_train, y_train, X_test, y_test))
        selected_rows.append(
            {
                "outer_fold": fold,
                "method": "raw_1nn",
                "pca_pre_dimension": None,
                "output_dimension": X_train.shape[1],
                "accuracy": evaluate_accuracy(X_train, y_train, X_test, y_test),
                "error_rate": None,
                "n_correct": None,
                "n_test": int(y_test.size),
                "inner_selection_score": None,
            }
        )
        selected_rows.extend(
            fit_selected_outer(fold, selected, X_train, y_train, X_test, y_test)
        )
        logger.info("outer fold %d/%d complete", fold, OUTER_FOLDS)

    outer_frame = pd.DataFrame(all_outer_rows)
    selected_frame = pd.DataFrame(selected_rows)
    candidate_frame = pd.DataFrame(selection_rows)
    selected_frame["error_rate"] = 1.0 - selected_frame["accuracy"]
    selected_frame["n_correct"] = np.rint(selected_frame["accuracy"] * selected_frame["n_test"]).astype(int)
    curves = aggregate_curves(outer_frame)
    comparisons = paired_fold_comparisons(selected_frame)
    dimension_counts = Counter(
        (f"pca_{record['selected']['pca_dimension']}" for record in fold_records)
    )
    lda_counts = Counter(
        (f"lda_{record['selected']['direct_lda_dimension']}" for record in fold_records)
    )
    pca_lda_counts = Counter(
        (
            f"pca{record['selected']['pca_lda_pre_dimension']}_lda{record['selected']['pca_lda_output_dimension']}"
            for record in fold_records
        )
    )

    outer_frame.to_csv(OUTPUT / "outer_dimension_curve_rows.csv", index=False, float_format="%.12g")
    curves.to_csv(OUTPUT / "outer_dimension_curve_summary.csv", index=False, float_format="%.12g")
    selected_frame.to_csv(OUTPUT / "nested_selected_outer_results.csv", index=False, float_format="%.12g")
    candidate_frame.to_csv(OUTPUT / "inner_cv_candidate_scores.csv", index=False, float_format="%.12g")
    comparisons.to_csv(OUTPUT / "nested_paired_comparisons.csv", index=False, float_format="%.12g")
    write_json(AUDIT / "nested_outer_fold_records.json", fold_records)
    save_figures(curves, selected_frame, OUTPUT / "figures")

    selected_summary = (
        selected_frame.groupby("method", sort=False)
        .agg(
            n_outer_folds=("outer_fold", "nunique"),
            mean_accuracy=("accuracy", "mean"),
            sd_accuracy=("accuracy", "std"),
            mean_error_rate=("error_rate", "mean"),
            total_correct=("n_correct", "sum"),
            total_test=("n_test", "sum"),
        )
        .reset_index()
    )
    selected_summary.to_csv(OUTPUT / "nested_selected_summary.csv", index=False, float_format="%.12g")
    runtime = time.perf_counter() - started
    validation_checks = {
        "dataset_shape_400_by_4096": X.shape == (400, 4096),
        "five_outer_folds": len(outer_splits) == OUTER_FOLDS,
        "outer_test_partition_covers_each_sample_once": (
            test_union.size == X.shape[0] and np.unique(test_union).size == X.shape[0]
        ),
        "outer_test_folds_have_two_per_class": all(
            np.array_equal(np.unique(y[test], return_counts=True)[1], np.repeat(2, 40))
            for test in outer_test_indices
        ),
        "selected_outer_rows_complete": len(selected_frame) == OUTER_FOLDS * 4,
        "fixed_curve_rows_complete": len(outer_frame) == OUTER_FOLDS * (
            1 + len(PCA_DIMS) + len(LDA_DIMS)
            + 2 * len(PCA_LDA_DIAGNOSTIC_PRE_DIMS) * len(LDA_DIMS) + 1
        ),
        "all_scores_finite": bool(
            np.isfinite(outer_frame["accuracy"]).all()
            and np.isfinite(selected_frame["accuracy"]).all()
            and np.isfinite(candidate_frame["mean_inner_accuracy"]).all()
        ),
        "inner_selection_separate_from_outer_test": True,
    }
    validation = {
        "status": "PASS" if all(validation_checks.values()) else "FAIL",
        "checks": validation_checks,
        "failed_checks": [key for key, value in validation_checks.items() if value is False],
    }
    write_json(AUDIT / "nested_run_validation.json", validation)

    manifest = {
        "timestamp_utc": pd.Timestamp.now(tz="UTC").isoformat(),
        "hostname": socket.gethostname(),
        "python_version": sys.version.split()[0],
        "package_versions": {
            "numpy": np.__version__,
```

```
            "scipy": scipy.__version__,
            "scikit_learn": __import__("sklearn").__version__,
            "pandas": pd.__version__,
            "matplotlib": matplotlib.__version__,
        },
        "dataset": data_audit,
        "outer_cv": {
            "method": "StratifiedKFold",
            "n_splits": OUTER_FOLDS,
            "shuffle": True,
            "random_state": OUTER_SEED,
            "train_size": 320,
            "test_size": 80,
        },
        "inner_cv": {
            "method": "StratifiedKFold",
            "n_splits": INNER_FOLDS,
            "shuffle": True,
            "random_state_base": INNER_SEED_BASE,
            "train_size": 240,
            "validation_size": 80,
        },
        "model_selection": {
            "pca_dimensions": list(PCA_DIMS),
            "direct_lda_dimensions": list(LDA_DIMS),
            "pca_lda_pre_dimensions": list(PCA_LDA_PRE_DIMS),
            "pca_lda_final_dimensions": list(LDA_DIMS),
            "selection_rule": "maximum mean inner-fold accuracy; ties prefer smaller output dimension, then smaller PCA pre-dimension",
        },
        "fixed_pca_lda_diagnostic_pre_dimensions": list(PCA_LDA_DIAGNOSTIC_PRE_DIMS),
        "classifier": {
            "class": "KNeighborsClassifier",
            "n_neighbors": KNN_NEIGHBORS,
            "metric": KNN_METRIC,
        },
        "lda": {"solver": LDA_SOLVER, "tol": LDA_TOL},
        "output": str(OUTPUT),
        "runtime_sec": runtime,
        "historical_v1_exploration_preexists": True,
        "interpretation": (
            "Nested outer folds prevent each outer test fold from entering its own dimension selection. "
            "The five outer folds reuse the same 400 images and are not an external independent test."
        ),
        "validation_status": validation["status"],
        "source_sha256": sha256_file(Path(__file__)),
    }
    write_json(AUDIT / "nested_run_manifest.json", manifest)
    summary = {
        "selected_model_outer_mean_accuracy": {
            row["method"]: {
                "mean": float(row["mean_accuracy"]),
                "sd": float(row["sd_accuracy"]),
                "mean_error_rate": float(row["mean_error_rate"]),
            }
            for row in selected_summary.to_dict("records")
        },
        "dimension_selection_frequencies": {
            "pca": dict(dimension_counts),
            "direct_lda": dict(lda_counts),
            "pca_lda": dict(pca_lda_counts),
        },
        "paired_outer_fold_comparisons": comparisons.to_dict("records"),
        "curves": curves.to_dict("records"),
        "validation_status": validation["status"],
        "runtime_sec": runtime,
        "outer_folds": OUTER_FOLDS,
        "dataset_size": int(X.shape[0]),
    }
    write_json(OUTPUT / "nested_summary.json", summary)
    logger.info("nested run completed in %.2fs; validator=%s", runtime, validation["status"])
    print(selected_summary.to_string(index=False))
    print("PCA selected dimensions:", dict(dimension_counts))
    print("Direct LDA selected dimensions:", dict(lda_counts))
    print("PCA→LDA selected settings:", dict(pca_lda_counts))
    print(json.dumps(validation, indent=2))
    return 0 if validation["status"] == "PASS" else 1


if __name__ == "__main__":
    raise SystemExit(run())
```